Customer SPA on messaging's member routes

Report of the work done on branch feat/gateway-api, 11 October 2026

Status	Done and verified; committed with this report.
Scope	The chat of the customer SPA (`gateway-api`: contact-seller button, inbox, thread) now runs against messaging as built, through the customer edge's existing routes and WebSocket token filter, instead of a sketch of the contract and local mock data. A way for a member to reach support was added. No edge or backend change.
Tests	50 SPA tests in 10 files pass (31 in 5 before); type check and translation parity (en, da, sv) green; the customer edge jar with the SPA built and driven headless with two members against messaging and announce-management; checklist in section 5.

1. What it is about

A member who reads an announce can message its seller. Messaging derives the conversation id from the announce and the buyer, so the button always lands in the same conversation. Inside it the two members exchange text, the buyer may make an OFFER (an amount in cents), and the seller accepts or declines it; an accepted offer is relayed as `OfferAcceptedV1` for the order module. A member can also open a SUPPORT conversation, which waits (WAITING) until a staff member joins from the admin queue wired in the previous round. Every conversation has a live WebSocket on `/ws/conversations/{id}`; the frames are the same for members and staff.

The customer screens were written on 26 September against the contract sketched in doc/order-context.md section 9, before the messaging module existed, and only ever ran on the service worker's fixtures. The customer edge has routed `/api/conversations/**` and `/ws/conversations/**` to messaging since then, with the service token and the query-token filter for the handshake, so the work was a comparison of the client with messaging as built, the corrections it called for, and a run against the service.

2. What the comparison found and what the SPA does now

Found	Now
Live frames mis-read: the socket pushes `messageId`, `offerAmountCents`, `currency`; the thread cast the frame to the REST row	A mapper turns the frame into the row shape before it is appended; de-duplication on message id, then client message id (the echo of an own post).
Own messages rendered as the other party's: the test was `senderType === 'MEMBER'`, messaging sends `USER`	`isMine`: USER plus the signed-in id. Staff lines carry a "Support" label, system lines are centred.
Open, accept, decline and close answer the aggregate, not the row; the thread stored it as the row	Typed as the aggregate answer; used for the id, the status and the offers; the row is refetched afterwards.
Composer only on OPEN, so a member could not write the first support message while WAITING	Writable on OPEN and WAITING; closed and blocked rows say why.
No way to open a SUPPORT conversation, so the admin queue was never fed from the product	The inbox has a "Contact support" form: subject and first message, then the thread.
Refetching the row on every frame raced the read-model projection (the live topic is tens of milliseconds ahead); the seller's accept box did not appear	The row is updated from the frame first (open offer from an OFFER message; status, joined staff, accepted offer, closed time from a status frame) and refetched once a second later.
Reading a just-opened conversation at once answered 404 until the projection wrote the row; the screen never recovered	The first misses are retried quietly every second; the alert shows after five.

Flags: `chat` defaults on; `trade` stays off because the trade screens still code against the section 8 sketch and were never checked against the order module. Mocks: the chat fixtures moved to `app/mocks/chatData.ts` in messaging's shapes and rules (deterministic listing ids, WAITING support rows, 403 not-participant, 409 rules, duplicate client message ids, no message posted by open, accept, decline or close); they serve the browser with `features.mocks` and the page tests.

3. What changed

Area	Files (under gateway-api/src/main/webapp)
Client and types	`app/shared/model/chat.ts` (messaging's views, requests, aggregate answer, frames, helpers), `app/shared/api/chatApi.ts`
Screens	`app/modules/chat/Thread.tsx`, `Inbox.tsx` (support form, polling), `ContactSeller.tsx`
Flags and mocks	`app/config/features.ts` (chat on), `app/mocks/chatData.ts` (new), `handlers.ts`, `data.ts` (trade only now), `public/mockServiceWorker.js` regenerated
Tests	`app/test/testUtils.tsx` and `fakeWebSocket.ts` (ported from the admin SPA), `jest.conf.js`, `jest.polyfills.js`, `app/config/store.ts` (`makeStore`), specs for Thread (9), Inbox (3), ContactSeller (3), chat helpers (4), features (2)
Dependencies	`msw` pinned 2.7.0 (2.15 pulls an ES-module dependency jest cannot load), `undici` 6.21.0 declared (undici 7 opens a MessagePort at load that keeps the jest worker alive)
Strings	`app/i18n/locales/{en,da,sv}/chat.json` rewritten: support form, status notices, offer states, header labels
Docs	`doc/frontend-edges.md` section 15 and 15a, pointers in sections 5 and 9; this report

4. Verification

- `npm test`: 50 tests in 10 files pass, no leaked worker. `npm run typecheck` and `npm run i18n:check` clean.
- Page tests run the real handlers over msw/node; axios goes through jsdom's XMLHttpRequest, which msw intercepts. The WebSocket double records the URL and the sent frames and lets a test push server frames.
- The customer edge jar was built with the webapp profile and driven headless (Playwright) with two freshly registered members in two browser contexts, the staff side over the admin edge's REST routes.

5. Checklist, executed 11 October 2026

Setup: live MySQL, Kafka 9094, customer edge on 8090 (routes pinned to announce-management 8085 and messaging 8086), admin edge on 8081 (dev profile), both backends from `spring-boot:run` with service tokens in enforce. Deviations: the development EventSourcingDB on port 3000 still hangs on streaming reads, so both backends ran against a throwaway store on port 3001 with the seven read-side checkpoint rows of messaging and announce-management deleted before and restored after; announce-management ran with the search indexer off; the members, the announce and the two conversations the run wrote were deleted from the development database afterwards.

Step	Result
Register seller and buyer; the seller posts and publishes a GOODS announce through the edge	201, 201, 201, 200; the row PUBLISHED with the seller's id
The seller opens their own announce	no "Message the seller" button
The buyer clicks "Message the seller"	201, thread opened, socket live on the edge with the token as query parameter; a second click lands in the same conversation
The buyer writes; the seller's inbox	post frame, echo as own bubble; inbox row "Ad … open with member #37", unread 1

Step	Result
The seller replies over the socket	on the buyer's screen in 157 ms
The buyer offers 4 200	offer frame in cents; the seller's Accept / Decline box in 137 ms (from the frame, before the row); the buyer sees the pending offer and no offer composer
The seller accepts	OFFER_ACCEPTED notice on the buyer's screen in 197 ms, offer line "(accepted)", box gone on the seller's side
The buyer contacts support from the inbox	SUPPORT conversation opened and first message posted; thread WAITING, composer enabled; row in the staff queue through the admin edge
Staff joins, replies, closes (admin edge)	JOINED notice and "Support agent #1" in 105 ms; staff line with the Support label in 108 ms; closed in 121 ms, composer disabled
Rules	post into the closed conversation 409 conversation-closed; the seller contacting their own announce 409 buyer-is-seller
Console	the expected 401 of the silent refresh before login, and one 404: the quiet row retry right after opening

6. Gotchas met on the way

- The shared ~/.m2 held a `cqrs-reactive` jar installed by another worktree without `security.Callers`; announce-management failed on the first POST with NoClassDefFoundError until `mvn -pl cqrs-reactive clean install` from this worktree and a restart.
- The customer login page has two submit buttons (header search first); a browser driver must click the login form's own button.
- `pkill -f` also matches the calling shell when the pattern appears later in the same command line; kill by saved PID or run the stop in its own command.
- undici 7 keeps a MessagePort open from load time, which jest reports as a leaked handle; the admin SPA's pin to 6.21.0 applies here too.

7. Left open

- An unread badge in the header (sum of the inbox).
- Image attachments (URLs only in messaging; no upload path in the thread).
- The customer trade screens against the order module (auction panel, buy box, orders), still behind `features.trade` off.
- The development EventSourcingDB on port 3000 needs a container restart before any backend can run against it again.